

Payment Services and Compliance Enforcement in a SCaaS from BPMN Models Solution

(* In <https://blockchain.cs.dal.ca/works/pubs/PaymentsCompliance.pdf>)

Consider the content in the following files/papers that you retrieved few chats back:

blockchain.cs.dal.ca/works/pubs/ACMj1.pdf

blockchain.cs.dal.ca/works/pubs/BCRAj1.pdf

blockchain.cs.dal.ca/works/pubs/BCRAj2.pdf

blockchain.cs.dal.ca/works/pubs/PaymentsCompliance.pdf

blockchain.cs.dal.ca/works/pubs/Payrails.pdf

blockchain.cs.dal.ca/works/pubs/Smart-Contracts-for-SMEs.pdf

*)

Abstract

Keywords:

1. Introduction

[§1] The publication of the Bitcoin white paper in 2008 (Nakamoto, 2008) and the subsequent launch of the Bitcoin network in 2009 sparked broad academic and commercial interest in blockchain technology. A particularly consequential development arising from that work was the introduction of programmable smart contracts—self-executing code deployed on a blockchain that automates agreements between mutually distrusting parties without the need for a central intermediary (Buterin, 2014). Smart contracts have since been applied to trade finance, supply chain management, decentralized finance (DeFi), and a growing range of multi-party collaborative processes. Yet, despite their promise, the development of correct, maintainable, and standards-compliant smart contracts remains a significant barrier to adoption (Kannengießer et al., 2021).

[§2] A central challenge is that writing smart contract code requires deep familiarity with blockchain platform intricacies—gas optimization, consensus semantics, on-chain storage costs, and language-specific constraints such as those imposed by Solidity on Ethereum or JavaScript on Hyperledger Fabric. These concerns are largely orthogonal to the business logic that domain experts wish to express, creating a gap between business intent and executable implementation (Mendling et al., 2018). The Model-Driven Engineering (MDE) paradigm has been proposed as a remedy: application requirements are first expressed at a high level of abstraction and then automatically transformed into executable artifacts, reducing manual developer effort and the risk of semantic divergence between specification and implementation (Weske, 2019; Mendling et al., 2018).

[§3] Business Process Model and Notation (BPMN), a standard developed by the Object Management Group (OMG), has emerged as a preferred modeling language for this purpose. BPMN is explicitly designed to be comprehensible to a wide range of stakeholders—including business analysts (BAs), managers, and technical developers—and naturally expresses conditional flows, parallelism, and multi-party collaboration (OMG, 2013). When augmented with Decision Model and Notation (DMN), which encodes business rules and decision logic in a structured, readable form using the Friendly Enough Expression Language (FEEL), BPMN models can represent both the control flow and the business logic of complex trade applications without requiring a developer to write platform-specific code (OMG, 2019). Please note that when we refer to DMN in the rest of the paper we also tacitly imply FEEL.

[§4] The Smart Contract as a Service (SCaaS) framework, developed at Dalhousie University (Bodorik et al., 2022; Liu et al., 2024), directly addresses this gap. In the SCaaS approach, a BA represents the business process

using BPMN with business logic expressed in DMN, and the SCaaS tool—known in earlier publications as Automatically Transforming BPMN Models to Smart Contracts (TABS)—automatically transforms that model into deployable smart contract methods. The transformation proceeds through an intermediate multi-modal representation that combines Discrete Event (DE) modeling for concurrency with Hierarchical State Machines (HSMs) to represent application functionality, collectively referred to as the DE-HSM model (Bodorik et al., 2022). A key architectural property of the DE-HSM model is its blockchain-agnostic coordination layer: the flow-of-control logic is expressed in platform-independent DE-HSM semantics, while only the implementation code for individual BPMN task elements must be written for a specific target platform (Bodorik et al., 2022). This separation allows the same BPMN model to target different blockchain platforms with minimal changes.

[§5] Subsequent work in the SCaaS research program has extended the core transformation methodology in several important directions. Liu et al. (2024) introduced TABS+, which augments the core BPMN-to-smart contract pipeline with support for nested collaborative transactions—complex multi-step activities that involve several actors and must satisfy transactional properties such as atomicity and consistency. Liu et al. (2025) then introduced TABS+R, which further extends this capability with mechanisms for smart contract upgrade and repair, when real-world conditions diverge from modeled assumptions, isolating affected BPMN fragments, modifying them, and regenerating consistent contract logic. Most recently, Liu et al. (2026) introduced a payment rails methodology within SCaaS that supports the identification and mapping of generic payment tasks in BPMN to pre-deployed payment smart contracts, enabling modular payment service reuse across on-chain, off-chain, and cross-chain contexts.

[§6] Yet, a specific and practically important class of payment pattern—multi-stage payments such as escrow and milestone-based payments—has not been addressed in detail within the SCaaS framework. In escrow-type arrangements, funds are held by a smart contract acting as a trusted intermediary and released only upon the satisfaction of predefined conditions (e.g., delivery confirmation or dispute resolution). In milestone-based payments, the total contracted sum is disbursed incrementally: each installment is released when the corresponding project stage has been completed and independently verified. Both patterns introduce sequencing constraints, conditional release logic, and compliance dependencies that go beyond simple payment invocation, and they are prevalent in trade finance, construction, freelance services, software development contracts, and international procurement. Supporting these patterns in a BA-accessible, model-driven manner requires extending the SCaaS methodology to handle staged fund custody, conditional release rules, and compliance gating as first-class modeling and generation concerns.

[§7] Compliance is a particularly critical dimension. In many regulated payment contexts, including international trade, anti-money laundering (AML) compliance, and know-your-customer (KYC) obligations, a payment may only legally proceed after an appropriate compliance certificate or clearance has been received from an authorized service. This requirement implies that compliance checking and payment execution must be logically separable (so that compliance rules can evolve independently) yet operationally integrated (so that payment is conditionally gated on compliance approval). The SCaaS framework's existing support for sidechain interactions and external service invocation provides a technical foundation for such integration, but the modeling conventions, BPMN fragment patterns, and code generation rules needed to make this accessible to BAs have not yet been specified.

[§8] This paper addresses these gaps by extending the SCaaS framework and its DE-HSM transformation methodology to support the generation of smart contracts for multi-stage payment services from BPMN models.

1.1 Objectives

This work pursues three main objectives:

1. Provide support for automated interaction with external services on the same blockchain, cross-chain, and off-chain—including bridge contracts and compliance certification services—within the BPMN-to-smart contract transformation pipeline.
2. Allow business analysts to represent multi-stage payment services—such as escrow or milestone-based payments—using BPMN modeling constructs and to automatically transform those representations into smart contracts. [No change.]
3. Ensure compliance rules are followed when performing payments: compliance and payment logic should be logically separate (representable as independently deployable smart contract services) but operationally integrated (so that a payment is permitted only after an appropriate compliance certificate has been received and verified on-chain). [No change.]

We included the first objective due to the widespread use of external and internal service by smart contracts. As services may be provided on the mainchain, cross-chain, and off-chain, we need a mechanism that allows BA to

represent invocations of service tasks using familiar BPMN constructs without concern for the underlying deployment topology.

1.2 Contributions

The contributions of this paper are as follows:

- *Support for external service interaction across on-chain, cross-chain, and off-chain contexts.* We extend the SCaaS tool generation pipeline to support invocation of external services in all three deployment contexts within the multi-stage payment workflow, including interaction with DeFi protocols, off-chain payment processors, and cross-chain bridge contracts, while maintaining the blockchain-agnostic coordination model that is a hallmark of the SCaaS approach.
- *BPMN modeling patterns for multi-stage payments.* We provide a mechanism to define a set of BPMN fragment patterns for multi-stage payments that are stored in a BPMN pattern repository. Patterns are retrieved by the BA and inserted into the BPMN model of the trade activity.
- *Transformation of multi-staged payment logic into smart contracts.* We describe how the BPMN fragment patterns for multi-stage payments are transformed into DE-HSM intermediate models and then to smart contracts, preserving the sequencing constraints, conditional release logic, and state isolation required for correct escrow and milestone behavior.
- *Compliance-gated payment generation.* We introduce a compliance integration pattern within the SCaaS framework whereby a smart contract generated from a BPMN model can invoke a separately deployed compliance certification service, on-chain, cross-chain, or off-chain, and condition payment execution on the receipt of a valid compliance certificate. The compliance contract and the payment contract are generated as separate, independently upgradeable services, reflecting the logical separation mandated by Objective 3.
- *Prototype implementation and evaluation.* We demonstrate the methodology through representative use cases, including a milestone-based payment scenario with AML compliance gating and an escrow payment, implemented and evaluated using the SCaaS tool. We evaluate correctness of state transitions using execution traces, and we discuss our approach's suitability for use by BAs.

1.3 Outline

[§9] The remainder of this paper is organized as follows. Section 2 provides background on BPMN modeling, the SCaaS/TABS methodology and DE-HSM transformation approach, and related work on BPMN-to-smart contract transformation and smart contract compliance. Section 3 discusses mechanism for the smart contract to use services provided by other smart contracts, which could be deployed on the same chain or cross-chain, or http services.

Section 4 describes the BPMN models for the two selected use cases, the escrow and milestone-based payments, both of which are stored in the repository. It also describes how a BPMN model of a staged payment is incorporated into the model of the trading activity, and how it is transformed into methods of a smart contract. Section 5 addresses the issue of compliance. It describes the methodology for ensuring that compliance enforcement is operationally integrated with payments, but also that enables separate management of payment methods and compliance rules. Section 6 discusses implications, limitations, directions for future work, and concluding remarks.

2. Background

...

... Background ...

...

3. Smart Contract Invocation of Services

[¶1] A smart contract generated by the SCaaS framework encodes the business logic of a multi-party trade process, but the execution of that process invariably depends on capabilities that lie outside the contract itself. A trade transaction may require locating and engaging a transporter, obtaining insurance coverage for goods in transit, verifying that the parties and the transaction satisfy applicable regulatory requirements, and ultimately executing a payment transfer. Each of these capabilities is provided by a distinct external service—whether a logistics platform, an insurance provider, a compliance authority, or a payment rail—that the generated smart contract must be able to invoke at the appropriate point in the process flow.

This section describes how the SCaaS framework supports such invocations, covering the architectural constraints that make external communication non-trivial, the approach adopted to overcome them, and the way in which service calls are represented in BPMN models and resolved at transformation time.

3.1 The Execution-Environment Constraint and the Invocation Approach

[¶2] Smart contracts execute within a tightly controlled, sandboxed environment. A deployed contract has no direct access to external variables, files, operating-system services, or network resources. This isolation is a deliberate security property of blockchain platforms: it ensures that execution is deterministic and tamper-resistant, because every participating node must reproduce exactly the same result from the same inputs. The isolation creates a practical challenge, however, when the outcome of a contract method must be communicated to the outside world or when the contract needs to trigger an action in an external system.

[¶3] Three strategies are commonly used to bridge this gap. The first is *state variable polling*: an external application repeatedly queries a blockchain state variable that the contract updates on completion. This is simple but inefficient, as it requires continuous retrieval and introduces latency. The second is *transaction log scanning*: the application inspects the event logs emitted by transactions, filtering for status events published by the contract. This is more efficient than polling but requires the application to maintain its own indexing layer. The third is *oracle-based relay*: a replicated, blockchain-aware service such as Chainlink reads off-chain data or listens for contract events and writes results back on-chain. Oracles are powerful but introduce external dependencies, replication overhead, and usage costs, which are undesirable when the target of the invocation is itself a smart contract.

In this work we adopt a fourth, more direct approach: *invocation via a helper smart contract*. When the target service is a smart contract—whether deployed on the same blockchain, on a different blockchain, or accessible through an off-chain HTTP endpoint via a gateway contract—the generated application contract invokes it directly, receiving a result either synchronously or asynchronously. This approach provides a unified invocation model that covers on-chain, cross-chain, and off-chain services without requiring a third-party relay.

[¶4] This invocation architecture builds on the service-reuse methodology introduced in the payment-rails SCaaS work (Liu et al., 2026), in which pre-deployed payment smart contracts are invoked from generated trade-activity contracts rather than re-implementing payment logic within each application. The present paper extends that principle to cover any type of external service that a smart contract may need to invoke during process execution. Furthermore, if the target pre-deployed smart contract does not exist in the repository, then it is generated directly from the BPMN model by the SCaaS transformation, and its information is stored in the repository.

The scope is broad: it includes logistics services such as transporter discovery, insurance services, regulatory and compliance verification services, and payment execution services. In every case the generated smart contract acts as an orchestrator that coordinates process execution while delegating specialised functionality to external service contracts.

3.2 Service Representation in BPMN and Deployment Parameters at Transformation Time

[¶5] A key design principle of the SCaaS approach to service invocation is the separation between what the business analyst models and what the deployment infrastructure requires. When constructing a process model, the BA represents each external service call as a BPMN Service Task (Object Management Group, 2013) identified by a label that reflects the business purpose of the call—for example, *FindTransporter* or *VerifyCompliance*. The data associations attached to the Service Task specify, in terms of the process data objects already present in the model, the inputs that will be passed to the service and the outputs that will be returned. Critically, *the BA does not embed any deployment information about the service endpoint or its deployment context in the BPMN model. Deployment context specifies whether a service resides on the main chain, on a cross-chain contract, or at an off-chain HTTP*

endpoint is entirely absent from the BPMN model. This keeps the BPMN model clean, portable, and comprehensible to business stakeholders who need not be aware of infrastructure concerns.

[¶6] The deployment information is supplied separately, at transformation time, when the BPMN model is submitted to the SCaaS tool for conversion into smart contracts. At that point, the SCaaS tool searches the repository for entries (which also contain the deployment information) that match the BPMN service task's input, output, and description. If multiple entries are matched, the BA determines which of the entries is to be for the deployment information of the service task. If no match is found, then the BA, or a system administrator with knowledge of the deployment environment, provides the parameters required for each Service Task: the network address of the target service, its deployment context (on-chain, cross-chain, or off-chain HTTP), and any additional platform-specific invocation metadata. The SCaaS tool combines these transformation-time parameters with the data associations declared in the BPMN model to generate the correct invocation logic in the resulting smart contract, including the appropriate inter-contract communication pattern—synchronous call, asynchronous request-callback, or HTTP relay—for each deployment context.

[¶7] Before generating invocation code for a Service Task, the SCaaS tool performs a validation step. Using the transformation-time parameters, it retrieves the meta-description of the target service—a structured record specifying the service's expected input parameters and output values, together with their types. The tool compares this meta-description against the data associations declared in the BPMN model. If they are consistent, transformation proceeds normally for that task element. If there is a mismatch, the element is flagged and the BA is prompted to correct either the BPMN model or the supplied transformation-time parameters; no invocation code is generated for the mismatched task. A mismatch on one Service Task does not prevent the tool from validating and transforming other correctly specified task elements in the same model.

Finally, we make an observation that for any service task deployed on the mainchain, i.e., the blockchain on which the trading activity smart contract is also deployed, invocation of such a service task is direct, without the use of a helper smart contract.

3.2 Illustrative Process with End-to-End Scenario

[¶9] To illustrate service invocation in the SCaaS framework, we use the trade transaction scenario from Liu et al. (2026), which models the sale of a large product and involves four external service invocations. In the BPMN model of the trading activity, each service call is represented as a BPMN *Service Task* element carrying a business-purpose label and the appropriate data associations. The address and deployment context of each target service are supplied when the model is submitted to the SCaaS tool for transformation.

3.2.1 BPMN Model and Deployment Information

The BPMN model of our sample use case as prepared by the BA is shown in Fig. 1. The first task, *RecAgr*, involves receiving an already approved purchase agreement, such as an approved purchase order. The agreement is represented by a BPMN data element, *SalesAgr*, associated with *RecAgr* task. The BA prepares the requests for insurance and transport represented by the task elements, *GetIns* and *GetTransp*, while describing information related to each task using a document associated with that task. The task *GetIns* obtains the insurance contract, represented by the *Insurance* associated document, which is then followed by the task element *GenPayIns* representing the generic payment for the insurance that was obtained.

The task *GetTransp* obtains a contract from a transporter for the product transport, with the contract being described in the *Transport* associated document. Obtaining and paying for insurance and obtaining the transporter can be executed concurrently, as indicated by the split parallel gateway (diamond shape with a plus sign). Once both insurance and transport contracts are obtained, the payment to the transporter is made as represented by the generic payment task *GenPayTrasnp*. The product is then received by the buyer, which is recorded in the *Delivery* associated document that is forwarded to the final task, *GenRcvePmnt*, that receives the payment from the buyer.

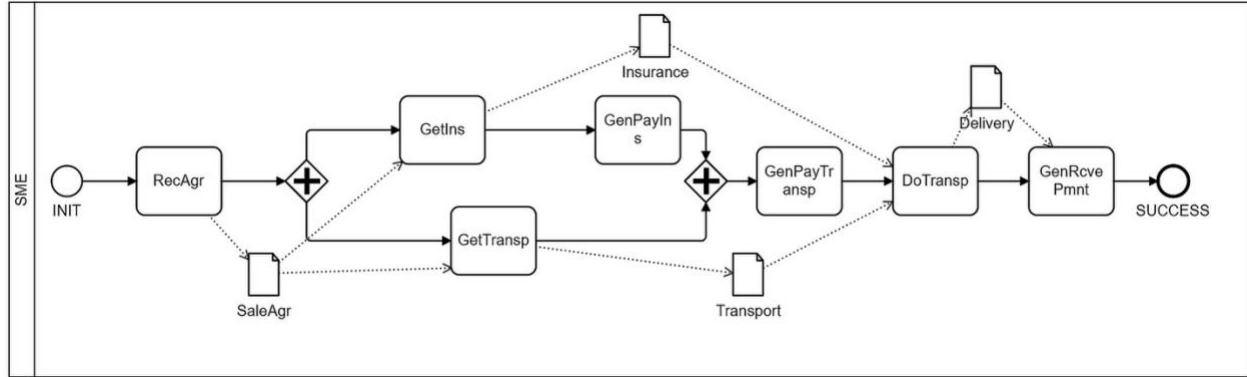


Fig. 1 – BPMN diagram of the sample use case
 (* Need changes to the BPMN model ... service tasks *)

As describe in the subsection 3.1, when creating the BPMN model of the trade activity, the BA specifies the task's logical name, input parameters received by the task and output parameters produced by the task, and the description of the service functionality. The SCaaS first uses this information against the existing entries in the repository of already generated service invocations to determine the service deployment information stored in the repository, the service address and deployment context. In case of no matches, the BA specifies the deployment information.

For our sample use case, we use the deployment information for the services implementing the service tasks and the generic payments shown in Table 1.

Table 1. Deployment information for service tasks and generic payments

(* Table entries are sample only and will need to be adjust to match the implementation *)

Task Name	Task type	Description	Deployment Quantifier	Service Address
<i>GetIns</i>	Service task	Obtain insurance contract for shipment	Off-chain	<code>https://insure-api.example.com/v1/getContract</code>
<i>GetTrasp</i>	Service task	Obtain transport contract from logistics provider	Off-chain	<code>0xA3f1...9C2B @ LogisticsChain (ChainID 1337)</code>
<i>GenPayIns</i>	Generic payment	Pay insurance premium (payable)	Cross-chain	<code>0xB7d2...4E1A @ USDxCrypto-StCn (ChainID 4200)</code>
<i>GenPayTrasp</i>	Generic payment	Pay transport provider (payable)	Cross-chain	<code>https://bankpay-api.example.com/v1/transfer</code>
<i>GenRcvePmnt</i>	Generic payment	Receive product payment from buyer (receivable)	On-chain	<code>0xC9e4...7F3D @ MainChain (ChainID 1)</code>

[98b] The table reflects the three deployment contexts supported by the SCaaS invocation mechanism. The *GetIns* service task invokes an off-chain insurance API via an HTTP endpoint. The *GetTrasp* service task invokes a cross-chain logistics contract on a separate blockchain. The generic payment *GenPayIns* is mapped to a cross-chain stablecoin payment service on the **USDxCrypto-StCn blockchain** (* to be changed*), consistent with the insurance provider's preferred payment rail. *GenPayTrasp* is mapped to an off-chain bank transfer service. *GenRcvePmnt*, which receives the buyer's payment, is mapped to an on-chain payment service that shares the mainchain ledger with the trade-activity smart contract, enabling atomic settlement. The deployment parameters provided here are supplied once at transformation time and embedded by the SCaaS tool in the generated smart contract invocation logic; the BA does not need to revisit them during subsequent executions of the deployed contract.

3.2.2 Execution traces using event logs

[¶9a] Our smart contract methods emit event messages to support monitoring of execution state and to provide evidence of correctness. Each event message is recorded on the blockchain as part of the transaction in which the corresponding smart contract method executes, creating a tamper-resistant, ordered log of execution progress. The event messages serve multiple purposes: they enable real-time monitoring of the trade activity's execution state, support manual verification of correct BPMN-to-smart contract transformation, facilitate debugging during development and testing, and provide the data needed to collect performance metrics such as per-invocation delay and total process completion time (ConsenSys, n.d.; Sei Foundation, 2025).

[¶9b] The event messages emitted during execution are displayed in a dedicated monitoring window, organized into four columns. The **first column** contains the name of the BPMN task element or event that executed. The **second column** lists the task or tasks that may execute next, corresponding to the outgoing sequence flows of the completed element. The **third column** shows the parameters used by the task, comprising the data that flows into the corresponding BPMN task element via its incoming data associations. The **fourth column** presents the result of the task, i.e., the content recorded on the blockchain as part of the transaction, including any outputs produced by the task and, for payment tasks, the payment receipt.

[¶9c] Figure 2 shows the event-message window for a single execution of the sample use case. The data shown in Figure 2 is given in Table 2 below.

Real-time Logs - Google Chrome			
about:blank			
Real-time Logs			
new_state	next_new_state	input	output
INIT	RecAgr	"N/A"	"N/A"
RecAgr	Gateway_0lg76nx	"initialization"	{ "productId": "product_identification", "maximumPrice": 100, "nextState": "GetTrReq" }
Gateway_0lg76nx	GetIns,GetTransp	"SaleAgr"	"N/A"
GetIns	GenPayIns	{ "productId": "product_identification", "maximumPrice": 100, "price": 98 }	Insurance
GetTransp	GenPayTransp	{ "productId": "product_identification", "maximumPrice": 100, "price": 98 }	Transport
GenPayIns	GenPayTransp	{ "fn": "Pay", "payer": "0x3a642c4ce0735549868cf1f0f153cdf2b6b58b84", "payee": "0xff6a862df7c85b5d20f08bccf6c95ba270d7f93", "amount": 10000000000000000 }	{ "txHash": "0x77ba980ecfac2032e869844876c45e1804820713490233e63d58fe1e5b44440", "status": true, "blockNumber": 1007246, "gasUsed": 54499, "event": "Pay", "payer": "0x3a642c4ce0735549868cf1f0f153cdf2b6b58b84", "payee": "0xff6a862df7c85b5d20f08bccf6c95ba270d7f93", "amount": 10000000000000000 }
GenPayTransp	DoTransp	{ "fn": "Pay", "payer": "0x03251bd2bd0e254d98db8b68440c422637d36945", "payee": "0x12039f15bd079bb2220bda47b2df6d379c583fad", "amount": 10000000000000000 }	{ "txHash": "0xe9f20f89586c21b02515fb3bd8b2070690230c02b2834b1efb9b54ae97e93f4d", "status": true, "blockNumber": 1003300, "gasUsed": 65846, "event": "Pay", "payer": "0x03251bd2bd0e254d98db8b68440c422637d36945", "payee": "0x12039f15bd079bb2220bda47b2df6d379c583fad", "amount": 10000000000000000 }
DoTransp	GenRcvePmnt	Insurance, Transport	Delivery
GenRcvePmnt	SUCCESS	{ "fn": "Pay", "payer": "0x5af29518e5171df3d7dc3ec27067a22c165081cb", "payee": "0xea43ed7825a662f647f12198485cc2da0ef63f1c", "amount": 10000000000000000 }	{ "txHash": "0x630338ae14081081507bf62c258071871dc62fa011928f0dbb365b6129f43e18", "status": true, "blockNumber": 1082724, "gasUsed": 43857, "event": "Pay", "payer": "0x5af29518e5171df3d7dc3ec27067a22c165081cb", "payee": "0xea43ed7825a662f647f12198485cc2da0ef63f1c", "amount": 10000000000000000 }
SUCCESS	N/A	Delivery Confirmation	"N/A"

Fig. 2. Window with event messages showing execution trace

[¶9d] The execution trace shown in Figure 2 confirms the correct transformation of the BPMN model of Figure 1 into a smart contract that faithfully reproduces the process control flow. The trace is interpreted as follows.

[¶9e] RecAgr. The first event message records the execution of the *RecAgr* task, which receives the purchase agreement and stores it as the *SalesAgr* data object on-chain. The Next Tasks column lists both *GetIns* and *GetTransp*, confirming that the parallel split gateway has been activated and that two concurrent execution tokens have been dispatched. This is consistent with the split parallel gateway (diamond with plus sign) immediately following *RecAgr* in the BPMN model.

[¶9f] GetIns and GetTransp (concurrent paths). The next two event messages record the execution of *GetIns* and *GetTransp*, which execute concurrently on the two parallel paths. *GetIns* invokes the off-chain insurance service and stores the returned insurance contract on-chain. *GetTransp* invokes the cross-chain logistics service and stores the returned transport contract on-chain. The parameters column for each task shows the data drawn from the *SalesAgr* document, confirming that the data associations declared in the BPMN model were correctly used to populate the service invocation calls at runtime. The Next Tasks column for *GetTransp* indicates *GenPayTransp*, but the annotation shows that the corresponding token is held pending the arrival of the token from the *GenPayIns* path—reflecting the join semantics described below.

[¶9g] GenPayIns (cross-chain payment). Following the completion of *GetIns*, the *GenPayIns* generic payment task executes, invoking the cross-chain stable-coin payment service at the USDxCrypto-StCn blockchain via the helper contract at address 0xB7d2...4E1A. The payment receipt—including the on-chain transaction identifier, confirmed status, and timestamp—is recorded on the mainchain. The Next Tasks column now shows *GenPayTransp* with both tokens present, indicating that the join condition has been satisfied: the token from the *GetTransp* path and the token from the *GenPayIns* path have both arrived. The *GenPayTransp* task may therefore begin execution.

[¶9h] Join gateway implementation. It should be noted that the parallel join gateway is not explicitly implemented as a separate construct in the generated smart contract. Instead, the SCaaS transformation routes the outgoing sequence flows of both *GetTransp* and *GenPayIns* directly as incoming flows to *GenPayTransp*. The *GenPayTransp* method is guarded by a condition that requires tokens to be present on all incoming sequence flows before execution proceeds. The trace confirms that this guard operates correctly: *GenPayTransp* does not appear in the event log until both concurrent paths have completed, consistent with the BPMN parallel join semantics.

[¶9i] GenPayTransp (off-chain payment). The *GenPayTransp* task invokes the off-chain bank transfer service via an HTTP endpoint, passing the transporter's bank account reference and the amount due. The callback from the payment processor confirms successful settlement, and the payment receipt is recorded on-chain. The Next Tasks column shows *GenRcvePmnt*, indicating that the process has advanced to the final sequential task.

[¶9j] GenRcvePmnt (on-chain payment receipt). The final event message records the execution of *GenRcvePmnt*, the receivable payment service. This task executes an atomic on-chain transfer of 85,000 USD from the buyer's mainchain wallet to the seller's, triggered upon receipt of the delivery confirmation document *DeliveryNote*. The transfer is settled within the same transaction context as the smart contract method invocation, consistent with the on-chain synchronous payment semantics described in Section 3.1. The Next Tasks column lists the BPMN End Event, confirming that the process has completed successfully.

[¶9k] Taken together, the six event messages confirm the following properties of the generated smart contract. First, the **sequential ordering** of *RecAgr* before the parallel split, *GenPayTransp* after the join, and *GenRcvePmnt* at the end matches the BPMN control flow exactly. Second, the **concurrent execution** of the two parallel paths—one through *GetIns* and *GenPayIns*, the other through *GetTransp*—is correctly reflected in the event log, with both tasks completing before *GenPayTransp* fires. Third, the **join guard** operates as required, holding *GenPayTransp* until both tokens are present. Fourth, the **data associations** declared in the BPMN model are correctly resolved at runtime: the parameters columns show that each service task and payment task received the inputs specified in the BPMN data associations, and the result columns confirm that the outputs were recorded on-chain as specified. Fifth, all three **deployment contexts**—off-chain, cross-chain, and on-chain—are exercised within a single process execution, and the generated invocation logic handles each correctly, including asynchronous callbacks for off-chain and cross-chain services and synchronous atomic execution for the on-chain service. These observations collectively verify the correctness of the BPMN-to-smart contract transformation for service task invocation as described in Section 3.1.

(* Fig. 2 contains event messages generated by execution of the smart contract, and thus it captures the flow of control in the BPMN model as it occurred during execution of our sample use case. The event messages indicate that the execution reached the *RecAgr* task, which is followed by a parallel split gateway. From this gateway, the flow of control is divided into two concurrent execution paths: one path consists of two sequential tasks, *GetIns* and *GenPayIns*, while the other path consists of the *GetTransp* task. *GetIns* and *GetTransp* are service tasks. The execution trace shows event messages received from the tasks *GetIns*, then *GetTransp*, and finally *GenPayIns*.

The flow of control from both concurrent paths subsequently reaches the exclusive join gateway that allows the flow to proceed only after both execution paths have completed (i.e., in BPMN terminology, when tokens from both incoming sequence flows arrive at the gateway). However, in our implementation of the transformation from a BPMN model to smart contracts, the join gateway is not explicitly implemented. Instead, we channel the incoming sequence flows to the gateway directly to the task that follows the join gateway, the *GenPayTransp* task. Of course, the *GenPayTransp* task execution starts only if the control tokens arrive on all incoming sequence flows to it. *)

(* Some concluding statements and comments on execution and what was done here. ... *)

4. Multi-staged Payments

[¶1] Multi-stage payments, or staged payments for short, differ fundamentally from single-stage payments in that the transfer of assets is not executed as a single, atomic operation but rather as a sequence of conditionally triggered stages that evolve with execution. Unlike single-stage payments, wherein the entire amount is transferred once basic preconditions such as invoice receipt or delivery confirmation are met, staged payments distribute obligations, risk exposure, and value realization across multiple phases of a business process. Each stage is associated with a distinct set of conditions, documents, or decision outcomes that must be satisfied before funds are released. Because these mechanisms directly govern the custody and transfer of funds, correctness is critical: any deviation from the intended semantics may result in financial loss, contractual disputes, or regulatory violations.

[¶2] A defining characteristic of staged payments is that funds may be partially transferred, temporarily withheld, or held in escrow pending verification of external events or approvals. Consequently, smart contracts implementing multi-stage payments must maintain persistent state across stages, accurately track progress, and correctly respond to inputs from participants or external services. Errors in state management or condition evaluation are particularly problematic: they may lead to premature release of funds, deadlock, or irreversible locking of assets.

[¶3] Common examples of multi-stage payment patterns include the following

- **Escrow payments**, in which the payer deposits funds into an escrow smart contract and the funds are released to the payee only after predefined conditions, such as delivery confirmation, inspection approval, or dispute resolution, are satisfied. Until those conditions are met, funds remain locked in the contract, reducing counterparty risk but requiring precise and verifiable release logic.
- **Milestone-based payments**, in which the total contract value is divided into installments corresponding to defined project milestones. Each installment is released only after evidence of milestone completion has been submitted and approved.
- **Retainage and holdback payments**, commonly used in construction and procurement, in which a percentage of each payment is withheld until final acceptance of the deliverable.
- **Phased service subscriptions or licensing fees**, in which payments are triggered periodically or upon renewal conditions, subject to performance and compliance checks.

[¶4] These patterns are prevalent in trade finance, construction, professional services, and international procurement, where trust between parties is limited and contractual obligations span extended periods. From a modeling and execution perspective, staged payments cannot be adequately captured as simple payment tasks. Instead, they require carefully designed and validated BPMN fragments that explicitly represent the full payment lifecycle, including initiation, conditional evaluation, staged execution, and completion or termination.

4.1 Modeling of Multi-stage Payments

[¶5] Given the critical importance of correctness in fund transfer, the SCaaS framework treats staged payments as verified, reusable payment patterns rather than ad hoc sequences of single-stage payments. Once a multi-stage payment pattern—such as an escrow or milestone-based payment—has been designed, verified, and validated, it is stored in a repository of reusable multi-stage payment fragments. These fragments encapsulate proven control-flow structures, decision logic, and payment semantics, and can be retrieved and reused by BAs when modeling trade activities. This repository-based approach reduces modeling errors, promotes consistency, and ensures that payment logic governing asset transfers is applied in a disciplined and reliable manner across applications.

[¶6] Each stage of a payment is represented as a BPMN task with an associated document that describes its input parameters, output parameters, and purpose. For each task representing a payment stage, the BA specifies constraints that must be satisfied before funds are released. Such constraints must identify:

- Which documents must be submitted, and by whom.
- Which FEEL expressions must evaluate to true before the stage proceeds. These expressions may reference:
 - Input parameters passed to the smart contract method.
 - The content of named fields in the documents specified as inputs or outputs.
 - Smart contract state variables.

[¶7] Expressions in the fragment patterns stored in the repository contain only representative values or placeholders, which guide the BA in supplying the actual values for a specific deployment. Examples of release constraint specifications are provided in the following subsection. The steps a BA follows when modeling a staged payment are shown in Fig. 5 and listed below.

Steps for modeling a staged payment (* Should be this in Fig. 5 *):

1. **Copy and integrate.** The BA retrieves the appropriate staged-payment BPMN fragment from the repository, copies it, and incorporates it into the trade-activity BPMN model.
2. **Associate documents.** For each task representing a payment stage, the BA associates a document specifying the task's input parameters, output parameters, and a description of the stage's purpose.
3. **Specify constraints using FEEL.** For each stage task, the BA enters the release constraints, specifically: (a) identifying the documents that must be submitted as preconditions; and (b) specifying the FEEL expressions that must evaluate to true before funds are released.

[¶8] Figure 6 shows sample BPMN fragments for escrow and milestone-based staged payments as stored in the repository. The escrow variant shown involves only the buyer depositing funds into the escrow account, as opposed to both buyer and seller staking funds. Each staged payment fragment also includes invocations of single-stage payment methods—as described in Section 3 and Liu et al. (2026)—to disburse funds to the payee once the release constraint for a given stage is satisfied. The staged payment logic governs *when* funds are released; the single-stage payment methods govern *how* the transfer is executed.

(**** Need an example of an escrow task constraint specification ****)

(*** Fig. BEGIN

(a) Sample escrow payment

(b) Sample milestone-based payment

Fig. 6. Sample escrow and milestone-based payments stored in the BPMN repository

Fig. 6 END *)

(* There should be a statement pointing out that staged payments also contain invocation of the single-staged payment methods to disperse funds when the specified constraints are satisfied *)

4.2 Transformation of the BPMN Model with Staged Payments

[¶9] Once BPMN fragments representing staged payments have been integrated into the trade-activity BPMN model, the SCaaS transformation pipeline proceeds following the standard SCaaS approach (Bodorik et al., 2022; Liu et al., 2024), with the addition of a step for generating staged payment APIs, as follows:

1. *DE-HSM conversion.* The BPMN model—now containing both the trade-activity logic and the staged-payment fragments—is transformed into a DE-HSM multi-modal model that captures control flow and execution state.
2. *Smart contract synthesis.* Using the SCaaS transformation mechanism, each DE-HSM sub-model is translated into smart contract methods. Invocation of single-stage payment methods to disburse funds is realized as described in Liu et al. (2026) and summarized in Section 3.2.
3. *Generation of staged APIs.* The BPMN model of a staged payment specifies the data that must be supplied when the API invokes the methods of each stage. The API must provide: (a) all inputs declared in the stage's BPMN data associations; (b) references to any documents that must be submitted as preconditions; and (c) the values of specific fields within those documents that are referenced by FEEL release constraints. The API is responsible for extracting the required field values from submitted documents before invoking the corresponding smart contract method.
4. *Deployment and runtime artifacts.* The generated smart contracts are deployed to the target blockchain, and the auxiliary APIs are produced to interface with the application layer.

4.3 Execution

[¶10] As in Section 3, we evaluate the correctness of the SCaaS transformation for staged payments by examining an execution trace produced during a representative run of the generated smart contract. We amend the base use case of Figure 1 by replacing the single-stage *GenRcvPmnt* receivable payment task with a two-stage escrow payment from

buyer to seller. The BA retrieves the escrow payment fragment from the BPMN repository and incorporates it into the trade-activity model, producing the intermediate BPMN model shown in Figure 6(a)

[¶11] The BA then completes the model by supplying the required information for each escrow stage. The finalized BPMN model is shown in Figure 6(b). The document associated with the *EscrowDep* task specifies a deposit amount of 85,000 USD. The document associated with the *EscrowRel* task specifies that funds may be released only if the document *RcveReport* has been submitted and the field *RcvdStatus* within that document carries the value "OK", as illustrated in Figure 7. Association documents are in JSON format.

[¶12] The execution trace for the escrow use case is shown in Figure 3. The trace data are presented in Table 3, using the same four-column format introduced in Section 3.2.2: executed BPMN task, next task or tasks, parameters, and result recorded on-chain. The trace and its verification discussion are **provided in full in the companion document for Section 4.3**.

5. Payment Compliance Enforcement

In section 5.1, we review those characteristics of payments and compliance enforcement that impact the architecture of compliance enforcement for payments, which is described in the subsection 5.2. The subsection 5.3 describes compliance as an end-to-end process, while the subsection 5.4 discusses approaches to reduce compliance enforcement overhead.

5.1 Characteristics of Payments and Compliance Enforcement

[¶1] In the context of smart contracts for the trade of goods and services, payment services and compliance enforcement represent two closely related but analytically distinct functions. Smart contracts enable automated payment services by executing transfers of value once predefined contractual conditions are satisfied—such as delivery confirmation or completion of contractual milestones. Compliance enforcement, by contrast, concerns the alignment of contractual performance with legal, regulatory, and policy requirements. Smart contracts can enforce compliance *a priori* to execution by embedding rule-based conditions that must be satisfied before payments or other contractual actions proceed. This encompasses both internal and external dimensions of compliance. Internal compliance refers to adherence to the contractual terms agreed upon by the parties, such as performance standards, delivery timelines, or usage restrictions. External compliance addresses obligations imposed by law or regulation, including know-your-customer (KYC) and anti-money laundering (AML) requirements, sanctions regimes, tax obligations, and sector-specific licensing rules. By encoding such requirements, smart contracts can automatically block, delay, or modify transactions that would otherwise produce non-compliant outcomes, while simultaneously generating an immutable audit trail.

[¶3] A particularly important characteristic distinguishing compliance enforcement from payment services is the *difference in their rates of change*. Payment services are mostly operationally stable: once a payment method has been deployed and validated, its core logic changes infrequently. Compliance rules, by contrast, are inherently dynamic. Regulatory obligations evolve continuously in response to geopolitical developments, legislative change, and enforcement priorities. Anti-money laundering (AML) screening lists are updated by their issuing authorities on unpredictable schedules, sometimes multiple times within a single day, and without advance notice. Counter-terrorist financing (CTF) designations and KYC requirements are similarly subject to change at short notice. A smart contract that encodes compliance rules directly in its payment logic would therefore need to be redeployed or upgraded every time a relevant regulation is amended, which is an operationally costly and error-prone process given the immutability of deployed contracts. This asymmetry between payment stability and compliance dynamism provides a strong architectural rationale for treating the two as *logically separate but operationally integrated* concerns.

5.2 Architecture for Compliance Enforcement using BPMN Augmentation

The above considerations lead to separation of logic for compliance and payments and at the same time to their operational integration. As compliance cannot be enforced directly by the smart contract due to its immutability and the dynamics of compliance rules, enforcement of compliance needs to be delegated to some service external the trade-activity smart contract.

To enforce payment compliance, we ensure that before each payment service is invoked, the compliance-enforcement is invoked, and the payment proceeds only if the compliance check allows it. This compliance check is achieved by specifying it explicitly in the BPMN model itself. For any generic payment in the BPMN model:

(i) A service task, which performs the compliance check, is inserted into the incoming sequence flow of the generic payment task.

(ii) A conditional-guard gateway is inserted between the service task and the generic payment that ensures the payment is executed only if the compliance check permits it.

Furthermore, this BPMM model augmentation is performed at the beginning of the transformation of the BPMN model into smart contracts – this is to ensure that the compliance checks are not amended inadvertently when the BPMN model is being modified by the BA. As with any other service task, the address and deployment context of the compliance service are supplied at transformation time.

Recall that the mechanism for invocation of a service represented by a service task, as described in the section 3, is such that, if the service is deployed as a smart contract on the mainchain, then it is invoked from the trade-activity smart contract directly for efficiency purpose, without the use of a helper smart contract. Thus, the compliance-check service task is deployed on the mainchain on which also the smart contract of the trade activity is deployed. This is to ensure efficient invocation of the compliance service task.

An alternative to BPMN augmentation for compliance is facilitating at transformation time invocation of the compliance check directly from the payment service itself, which would be opaque in the BPMN model context. However, representing the compliance check at the BPMN model is important for *review and audit purposes*: The trade activity logic is represented using BPMN that is understandable by business as well as IT personnel. Thus, it allows *tracing code logic from business processes* and thus facilitates auditable lineage from requirements to code that results in easier regulatory validation.

In addition, we also facilitate monitoring of the trade activity’s execution progress by smart contracts sending event notifications to the monitor process. The monitor process receives the notifications and maps them to the BPMN model to show the current state of execution of the trade activity using a coloring scheme identifying completed parts of the BPMN model.

5.2 Compliance Service Task

The BPMN compliance service task is transformed into invocation of a method of a separate smart contract that performs the actual compliance checks, a smart contract that is reusable and thus is used for any before payment service of any trade activity smart contract. As the compliance checking is modeled within SCaaS as a service, it is invoked using the same unified service invocation mechanism described in Section 3. Furthermore, it is deployed on the mainchain for efficient invocation by the trade activity.

The result of each compliance check, whether pass or fail, is recorded on-chain as a compliance proof, providing an auditable, tamper-resistant record. Describe the structure of the compliance proof (service identifier, timestamp, inputs checked, outcome, reference to any supporting data stored off-chain via IPFS).

However, implementation of the service itself is out of scope of this paper as it depends on evaluating trade-offs between risks and benefits associated with the way checks are performed. For instance, external regulations mandate assurance that a payment is not made to an organization that is on an externally managed AML and anti-terrorists lists, which are updated at irregular intervals, perhaps several times a day. Checking such lists incurs not only overhead cost due to invoking externally provided services, whether cross-chain or off-chain, but also due to the chargers that the service itself makes. If there are many payments made, costs may be reduced by keeping the lists cached within the compliance smart contract and check each payment against the cached lists instead of invoking external methods. The cached lists would have to be updated with some frequency, which leads to be evaluation of cost reduction benefits vs. the risk of checking the payment against the cached list that is outdated, resulting in a non-compliant payment.

5.3 Execution

We extended the SCaaS tool generation pipeline to support compliance, and we use our sample use-case shown in Fig. 1 to verify the correctness of our approach. Once a BA completes the BPMN model and initiates its transformation into smart contracts and API, before transformation proceeds, the BPMN model is augmented with compliance checks resulting in the BPMN model shown in Fig. 7. Each payment is preceded in the BPMN model by invocation of the service task. The augmented model is transformed into the smart contracts and APIs.

For demonstration purposes, we used the same deployment information for service tasks and generic payments as shown in Table 1. In addition, the compliance checking service was deployed on the mainchain (with address

0xA3f1...9C2B). The execution trace is shown in Fig. 8 confirming the correct transformation of the BPMN model of Figure 7 into a smart contract that faithfully reproduces the process control flow. The figure shows that before each payment service event log message, there is a log message generated by the compliance smart contract that check the request and generated the compliance proof ID, with the content of the compliance proofs recorded on IPFS.

Fig. 7. BPMN model with compliance check augmentation

5. Payment Compliance Enforcement

[¶1] In Section 5.1, we review those characteristics of payments and compliance enforcement that bear on the architecture of compliance enforcement for payments, which is described in Section 5.2. Section 5.3 describes the compliance service and the compliance proof it generates. Section 5.4 presents the execution trace that verifies the correctness of the approach.

5.1 Characteristics of Payments and Compliance Enforcement

[¶2] In the context of smart contracts for the trade of goods and services, payment services and compliance enforcement represent two closely related but analytically distinct functions. Smart contracts enable automated payment services by executing transfers of value once predefined contractual conditions are satisfied—such as delivery confirmation or completion of contractual milestones. Compliance enforcement, by contrast, concerns the alignment of contractual performance with legal, regulatory, and policy requirements. Smart contracts can enforce compliance by a priori by embedding rule-based conditions that must be satisfied before payments or other contractual actions proceed. This encompasses both internal and external dimensions of compliance. Internal compliance refers to adherence to the contractual terms agreed upon by the parties, such as performance standards, delivery timelines, or usage restrictions. External compliance addresses obligations imposed by law or regulation, including know-your-customer (KYC) and anti-money laundering (AML) requirements, sanctions regimes, tax obligations, and sector-specific licensing rules. By encoding such requirements, smart contracts can automatically block, delay, or modify transactions that would otherwise produce non-compliant outcomes, while simultaneously generating an immutable audit trail.

[¶3] A particularly important characteristic distinguishing compliance enforcement from payment services is the difference in their *rates of change*. Payment services are operationally stable: once a payment method has been deployed and validated, its core logic changes infrequently, if at all. The mechanics of transferring value between parties are well-defined and do not respond to rapidly shifting external events. Compliance rules, by contrast, are inherently dynamic. Regulatory obligations evolve continuously in response to geopolitical developments, legislative change, and enforcement priorities. Anti-money laundering screening lists—such as the U.S. Office of Foreign Assets Control (OFAC) Specially Designated Nationals (SDN) list and the EU Consolidated Financial Sanctions List—are updated by their issuing authorities on unpredictable schedules, sometimes multiple times within a single day and without advance notice. Counter-terrorist financing (CTF) designations and KYC requirements are similarly subject to change at short notice. A smart contract that encodes compliance rules directly in its payment logic would therefore need to be redeployed or upgraded every time a relevant regulation is amended—an operationally costly and error-prone process given the immutability of deployed contracts. This asymmetry between payment stability and compliance dynamism provides a strong architectural rationale for treating the two as *logically separate but operationally integrated concerns*. Payment services are primarily concerned with the efficient and reliable transfer of value, whereas compliance enforcement is oriented toward legality, risk mitigation, and accountability. Maintaining this separation supports modular system design: compliance rules can be updated or replaced without modifying or redeploying the core payment logic.

At the same time, operational integration is essential: compliance conditions must directly govern payment execution in order to be effective. Accordingly, the SCaaS framework adopts a layered approach in which payment mechanisms and compliance controls remain distinct in purpose but are seamlessly integrated at the point of execution. The following subsections describe how this is achieved.

5.2 Architecture for Payment Compliance Enforcement using BPMN Augmentation

[¶4] The considerations of Section 5.1 lead to a separation of compliance and payment logic while simultaneously achieving their operational integration. Because the trade-activity smart contract is immutable once deployed and compliance rules change dynamically, compliance enforcement cannot be embedded directly in the trade-activity contract; it must instead be delegated to a service that is external to it and independently upgradeable.

[¶5] To enforce payment compliance, the SCaaS framework ensures that the compliance-enforcement service is invoked before each payment service, and that the payment proceeds only if the compliance check returns a positive result. This compliance check is made explicit by specifying it in the BPMN model itself. For any generic payment in the BPMN model, the following two elements are inserted:

- (i) A Service Task that performs the compliance check is inserted into the incoming sequence flow of the generic payment task.
- (ii) A conditional gateway is inserted between the compliance Service Task and the generic payment task, ensuring that payment is executed only if the compliance check returns a positive result.

[¶7] This BPMN model augmentation is performed at the beginning of the transformation process, before the BA's BPMN model is submitted for DE-HSM conversion. Performing augmentation at this stage ensures that compliance checks are not inadvertently omitted or modified if the BPMN model is subsequently edited. As with any other Service Task, the address and deployment context of the compliance service are supplied at transformation time.

[¶8] As described in Section 3, when a target service is deployed as a smart contract on the same blockchain as the trade-activity contract, the SCaaS framework invokes it directly, without the use of a helper smart contract, for efficiency. Accordingly, the compliance-check service is deployed on the same mainchain as the trade-activity smart contract. This co-deployment ensures that compliance checks are invoked with minimal latency and without the additional overhead of cross-chain or off-chain relay.

[¶9] An alternative to BPMN augmentation is to invoke the compliance check directly from the payment service smart contract at transformation time, without representing it in the BPMN model. This alternative is rejected for two reasons. First, compliance would be invisible in the BPMN model: neither business stakeholders nor auditors could determine from the model that compliance checks are in place. This matters because the SCaaS framework uses BPMN as the authoritative, human-readable representation of trade-activity logic from which executable smart contract code is generated—an auditor must be able to trace compliance requirements directly from the BPMN model to the generated code in order to validate regulatory conformance. Second, BPMN-layer compliance representation is required to support execution monitoring: since the compliance Service Task emits an event notification like any other task, the monitor process can reflect compliance checks in the process-state display alongside other task executions.

5.2 Compliance Service Task

[¶11] The BPMN compliance Service Task is transformed by the SCaaS tool into an invocation of a dedicated compliance smart contract that performs the actual compliance checks. This contract is reusable: once deployed, it can be invoked by the trade-activity smart contract of any trade process that requires compliance gating before payment. As compliance checking is modeled within SCaaS as a service, it is invoked using the same unified service invocation mechanism described in Section 3. The compliance service is deployed on the mainchain for efficient invocation by the trade-activity smart contract.

[¶12] The result of each compliance check—whether a pass or a fail—is recorded on-chain as a compliance proof, providing an auditable and tamper-resistant record. Each compliance proof contains the following fields: (a) a service identifier, identifying the compliance smart contract and the specific rule set evaluated; (b) a timestamp, recording when the check was performed; (c) the inputs checked, expressed as the parameters passed to the compliance service method; (d) the outcome (pass or fail); and (e) a content-addressed reference to any supporting data stored off-chain via IPFS, such as the specific list entry or regulatory clause that triggered a failure.

[¶13] Implementation of the compliance service itself is outside the scope of this paper, as it requires evaluating trade-offs between cost and compliance risk that depend on the operational context. A representative example concerns external regulatory lists, such as AML screening lists and anti-terrorist financing designation lists, which are maintained by external authorities and updated at irregular intervals—potentially several times per day. Invoking an external list-checking service on each payment check incurs both communication overhead and the service charges levied by the external provider. When payment volumes are high, costs can be reduced by caching the

current list within the compliance smart contract and checking each payment against the cached copy rather than invoking the external service on every occasion. However, a cached list may be outdated at the time of the check, creating a risk that a non-compliant payment passes undetected. The appropriate balance between cost reduction and compliance risk is a deployment-level decision that falls outside the scope of this paper.

5.3 Execution

[¶14] We extended the SCaaS tool generation pipeline to support compliance enforcement and use the sample use case of Figure 1 to verify the correctness of the approach. Once the BA completes the BPMN model and initiates its transformation into smart contracts and APIs, the SCaaS tool first augments the BPMN model with compliance checks as described in Section 5.2, producing the augmented BPMN model shown in Figure 7. In the augmented model, each generic payment task is preceded by a compliance Service Task and a conditional gateway. The augmented model is then transformed into smart contracts and APIs following the standard SCaaS pipeline.

[¶15] For demonstration purposes, the same deployment information for service tasks and generic payments as shown in Table 1 is used. In addition, the compliance checking service is deployed on the mainchain at address 0xA3f1...9C2B. The execution trace is shown in Figure 8. The trace confirms the correct transformation of the augmented BPMN model of Figure 7 into a smart contract that faithfully reproduces the specified control flow. In the trace, each payment-service event log entry is preceded by an event log entry generated by the compliance smart contract, which records the compliance check request and the identifier of the resulting compliance proof. The content of each compliance proof—including the inputs checked, the outcome, and any supporting regulatory data—is stored on IPFS, and the proof identifier recorded on-chain serves as a tamper-resistant, content-addressed reference to that off-chain record.

6. Related Work, Limitations, Future Work, and Concluding Remarks

...

References

- Bodorik, P., Liu, C. G., & Jutla, D. (2022). TABS: Transforming automatically BPMN models into blockchain smart contracts. **Blockchain: Research and Applications**, *3*(4), 100067. <https://doi.org/10.1016/j.bcra.2022.100067>
- Buterin, V. (2014). **A next generation smart contract & decentralized application platform** [White paper]. Ethereum Foundation. <https://ethereum.org/en/whitepaper/>
- Liu, C. G., Bodorik, P., & Jutla, D. (2021). Smart contracts for SMEs and large companies. In **Proceedings of the 2021 IEEE Virtual Conference on Communications (VCC)**. IEEE. <https://doi.org/10.1109/VCC63113.2024.10914418>
- Liu, C. G., Bodorik, P., & Jutla, D. (2024). Transforming automatically BPMN models to smart contracts with nested collaborative transactions (TABS+). **ACM Journal on Distributed Ledger Technologies: Research and Practice**, *3*(3), Article 21. <https://doi.org/10.1145/3654802>
- Liu, C. G., Bodorik, P., & Jutla, D. (2025a). Automated mechanism to support trade transactions in smart contracts with upgrade and repair. **Blockchain: Research and Applications**. **Pre-accepted**. <https://blockchain.cs.dal.ca/papers/BCRAj2>
- Liu, C. G., Bodorik, P., & Jutla, D. (2025b). BPMN to smart contract by business analyst. **arXiv preprint arXiv:2505.22612**. <https://arxiv.org/abs/2505.22612>
- Liu, C. G., Bodorik, P., & Jutla, D. (2026). Payment rails in Smart Contract as a Service (SCaaS) solutions from BPMN models. **Future Internet**, *18*(2), 110. <https://doi.org/10.3390/fi18020110>
- Mendling, J., Weber, I., van der Aalst, W., vom Brocke, J., Cabanillas, C., Daniel, F., Debois, S., Di Ciccio, C., Dumas, M., Dustdar, S., Gal, A., Garc√a-Ba√uelos, L., Governatori, G., Hull, R., La Rosa, M., Leopold, H.,

Leymann, F., Recker, J., Reichert, M., & Zhu, L. (2018). Blockchains for business process management: Challenges and opportunities. *ACM Transactions on Management Information Systems*, 39(1), 4. <https://doi.org/10.1145/3183367>

Nakamoto, S. (2008). *Bitcoin: A peer-to-peer electronic cash system* [White paper]. <https://bitcoin.org/bitcoin.pdf>

Kannengießer, N., Lins, S., Sander, C., Winter, K., Frey, H., & Sunyaev, A. (2021). Challenges and common solutions in smart contract development. *IEEE Transactions on Software Engineering*, 48(11), 4291–4318. <https://doi.org/10.1109/TSE.2021.3116808>

Object Management Group. (2013). *Business Process Model and Notation (BPMN) version 2.0.2* (formal/2013-12-09). OMG. <https://www.omg.org/spec/BPMN/2.0.2>

Object Management Group. (2019). *Decision Model and Notation (DMN) version 1.3* (formal/2019-12-08). OMG. <https://www.omg.org/spec/DMN/1.3>

Weske, M. (2019). *Business process management: Concepts, languages, architectures* (3rd ed.). Springer. <https://doi.org/10.1007/978-3-662-59432-2>

Zou, W., Lo, D., Kochhar, P. S., Le, X.-B. D., Xia, X., Feng, Y., Chen, Z., & Xu, B. (2021). Smart contract development: Challenges and opportunities. *IEEE Transactions on Software Engineering*, 47(8), 2084–2106. <https://doi.org/10.1109/TSE.2019.2942301>

&&& *** Renive Zou reference as citation has been removed?

References

Chainlink. (n.d.). *Smart contracts and external data: The complete guide*. <https://chain.link/article/smart-contracts-external-data>

Chainlink Documentation. (n.d.). *Smart contract overview*. <https://docs.chain.link/getting-started/conceptual-overview>

ConsenSys. (n.d.). *Guide to events and logs in Ethereum smart contracts*. <https://consensys.io/blog/guide-to-events-and-logs-in-ethereum-smart-contracts>

Gemini. (n.d.). *Chainlink oracle network: Smart contracts with LINK*. <https://www.gemini.com/cryptopedia/chainlink-crypto-oracle-link-token>

Sei Foundation. (2025). *What are event logs and how do they work?* <https://blog.sei.io/blockchain/fundamentals/what-are-event-logs-and-how-do-they-work/>

...

(*** to be included somewhere

- Modeling is carried out by a Business Analyst (BA) who is assumed to be proficient in BPMN and DMN modeling, including the use of the FEEL language for decision logic. Additionally, we assume that the BA is familiar with JavaScript Object Notation (JSON), which is used to describe the flow of information throughout the computation process, as will be detailed later.
- Use of AI in creating models and in specifying functionality of tasks.

***)

(* ??? need to say that if the service task is on the mainchain, helper service task is not invoked, rather direct invocation is used from the main smart contract [¶8] The same mechanism applies directly to compliance verification services, which represent an important class of external service in regulated payment contexts. In many jurisdictions—covering international trade, anti-money laundering (AML) enforcement, and know-your-customer (KYC) screening, among others—a payment may not legally proceed until an authorised compliance service has confirmed that the transaction and the parties involved satisfy the applicable regulatory requirements. Within the SCaaS framework, such a compliance check is modeled as a BPMN Service Task positioned before the payment task in the process flow, with data associations capturing the information to be verified and the certificate or clearance result to be returned. As with any other Service Task, the address and deployment context of the compliance service are supplied at transformation time. The compliance result is recorded on-chain, providing an auditable proof of compliance, and its value determines—via the decision logic expressed in DMN—whether process execution may proceed to the payment step. The detailed design of compliance gating, including the separation of compliance rules from payment logic and the mechanisms for independent management of each, is addressed in Section 5. **** ??? *)

(*

- The SCaaS tool matches each generic payment meta-data of the BPMN generic payment task with the meta-data of the executable payment services stored in the repository. In case of multiple matches, the BA determines which payment service is to be used. Following this, the matched payment service invoked by the trade activity smart contract to perform the payment (Liu et al., 2026).
- The executable payment services themselves are prepared by the developer and are stored in the repository. Correspondingly, its meta-data description consists of service's input and output parameters, and a description of the service purpose.

*)

(*

However, implementation of the service itself is out of scope of this paper as it depends on evaluating trade-offs between risks and benefits associated with the way checks are performed. For instance, external regulations mandate assurance that a payment is not made to an organization that is on an externally managed AML and anti-terrorists lists, which are updated at irregular intervals, perhaps several times a day. Checking such lists incurs not only overhead cost due to invoking externally provided services, whether cross-chain or off-chain, but also due to the charges that the service itself makes. If there are many payments made, costs may be reduced by keeping the lists cached within the compliance smart contract and check each payment against the cached lists instead of invoking external methods. The cached lists would have to be updated with some frequency, which leads to be evaluation of benefit, cost reduction, vs. the risk of checking the payment against the cached list that is outdated, resulting in a non-compliant payment.

*)

(* Recall that in SCaaS, a payment is represented in a BPMN model as a generic payment task with parameters that specify semantic information, such as a payer, payee, invoice ID or some other indication of the payment's purpose, commodity/product classification or ID, but without implementation details. The meta-data information of a BPMN element consists of input data/objects flowing into the service task along the incoming sequence flow and flowing out of the service task along the outgoing sequence flow, plus a description of the tasks purpose.

*)